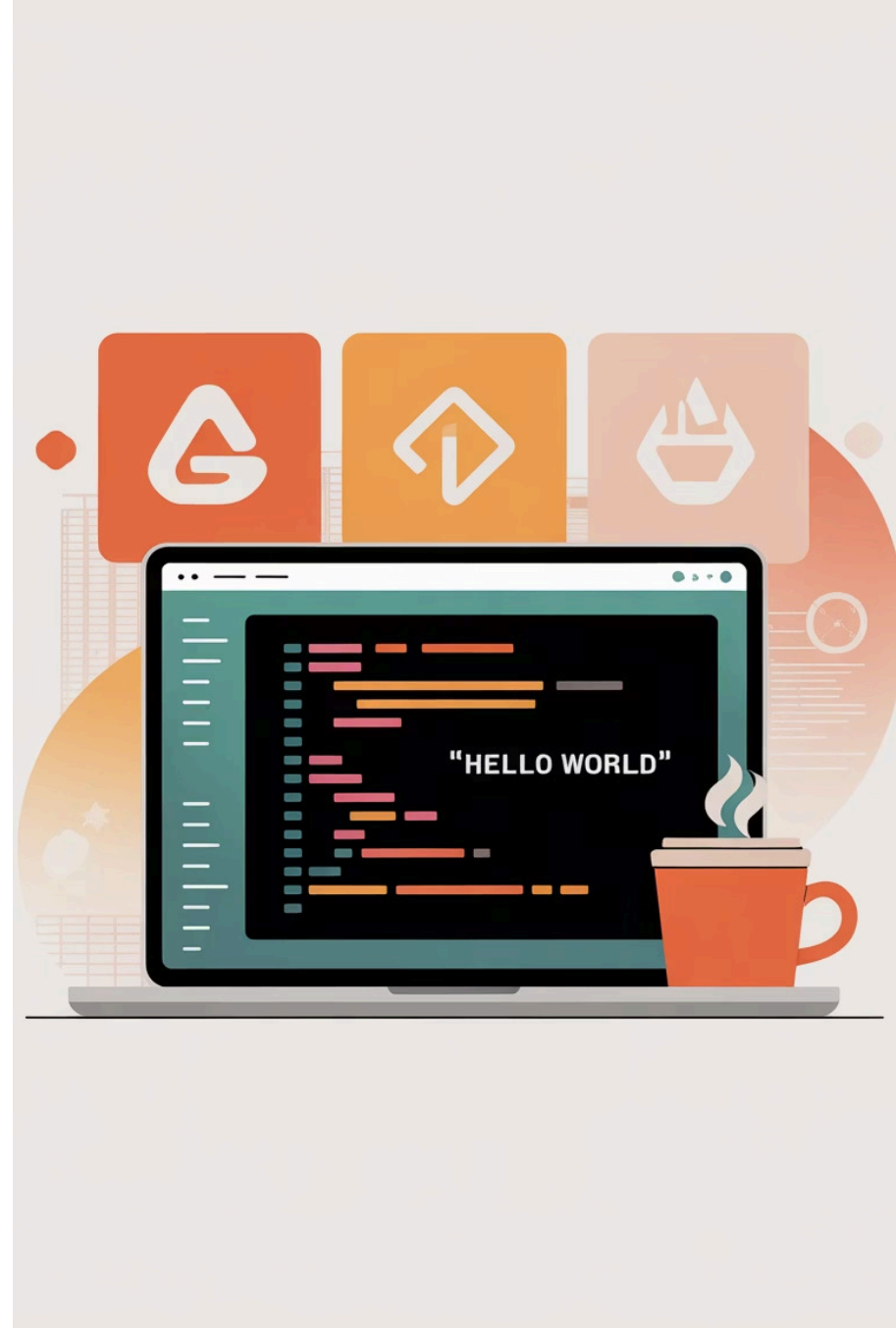


ApplicationContext y Beans

Java con Spring Boot de 0 a Pro • Fundamentos de Spring Boot •
Video #3



¿Qué aprenderás hoy?

1

Qué es el ApplicationContext

El contenedor central que maneja toda tu aplicación Spring

3

Detección y gestión de componentes

Cómo Spring encuentra y registra tus clases

2

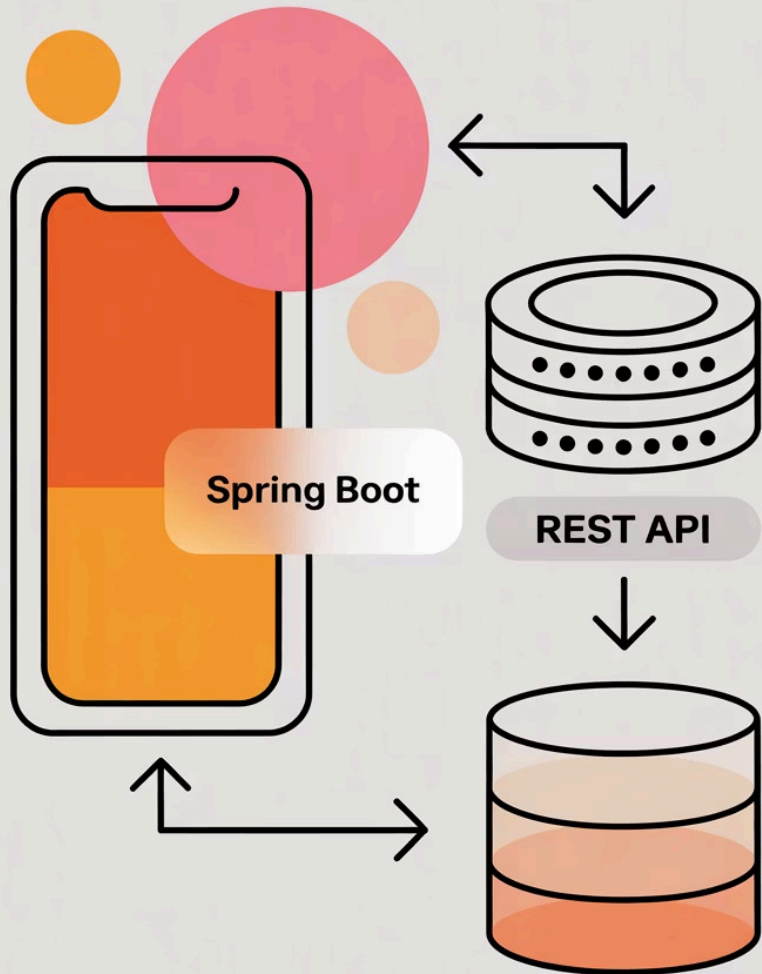
Concepto de Bean y su importancia

Los objetos que Spring gestiona automáticamente por ti

4

Ciclo de vida básico de un Bean

Desde su creación hasta su destrucción



Agenda del video

01

Conceptos fundamentales

ApplicationContext y Beans explicados

02

Detección automática

Component scan en acción

03

Ciclo de vida

Cómo nacen y viven los beans

04

Demo práctica

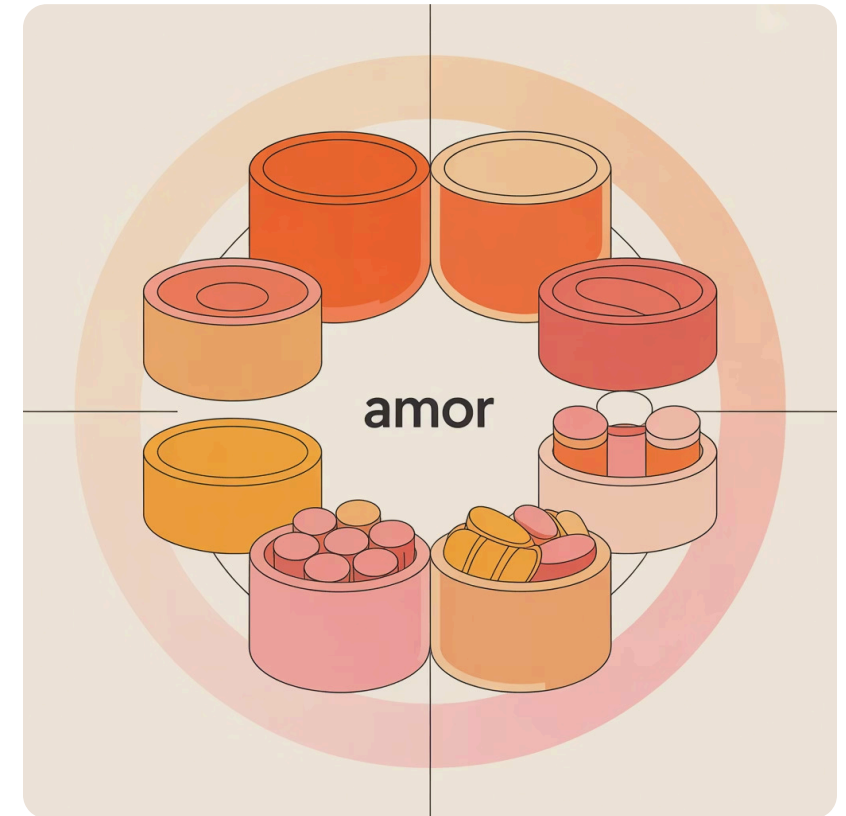
Código en vivo para verlo funcionar

¿Qué es ApplicationContext?

El **ApplicationContext** es el corazón de Spring Boot. Piénsalo como el **administrador principal** de tu aplicación.

- Contenedor central que gestiona todos los objetos
- Se encarga de crear, configurar y conectar componentes
- Mantiene el registro de todos los beans disponibles
- Proporciona servicios como inyección de dependencias

Es como tener un **mayordomo súper inteligente** que sabe exactamente qué necesitas y cuándo lo necesitas.





ApplicationContext: La caja mágica

Imagina el ApplicationContext como una **caja inteligente** que guarda y organiza todos tus objetos

Almacena

Guarda todos los beans de tu aplicación de forma ordenada

Conecta

Relaciona automáticamente los objetos que se necesitan entre sí

Administra

Controla cuándo crear, usar y destruir cada objeto

¿Qué es un Bean?

Un **Bean** es simplemente un objeto que Spring gestiona por ti. En lugar de crear objetos con `new`, dejas que Spring los cree y administre automáticamente.

- Cualquier clase puede convertirse en un bean
- Spring se encarga de su ciclo de vida completo
- Los beans viven dentro del ApplicationContext
- Son reutilizables en toda la aplicación



Ejemplo de Bean

Cualquier clase de tu aplicación puede ser un bean. Aquí tienes ejemplos típicos:



Servicios

Clases que contienen la lógica de negocio de tu aplicación, como `UsuarioService` o `PedidoService`



Repositorios

Componentes que manejan el acceso a datos, como `UsuarioRepository` para interactuar con la base de datos



Controladores

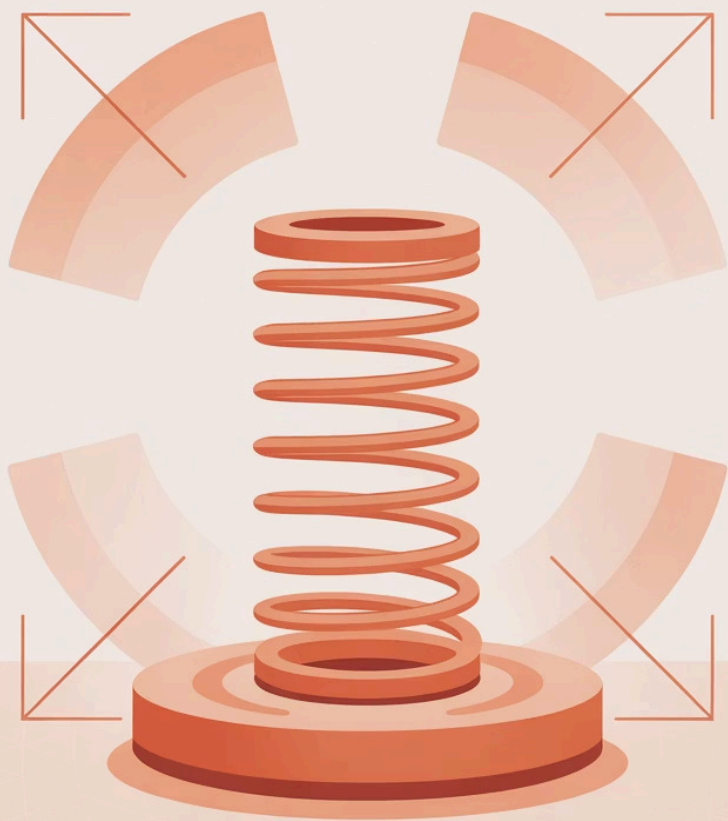
Clases que manejan las peticiones HTTP, como `UsuarioController` para crear endpoints REST



Configuraciones

Clases de configuración que definen otros beans o ajustes específicos de la aplicación

Lo importante es que **Spring los crea una vez y los reutiliza** cada vez que los necesites.



¿Cómo Spring detecta beans?

Component Scan: El detective automático



Escanea

Spring busca clases con anotaciones especiales como `@Component`



Registra

Las clases encontradas se registran automáticamente como beans



Almacena

Los beans quedan disponibles en el ApplicationContext



Consejo: Por defecto, Spring escanea desde el paquete donde está tu clase principal (`@SpringBootApplication`) hacia abajo.

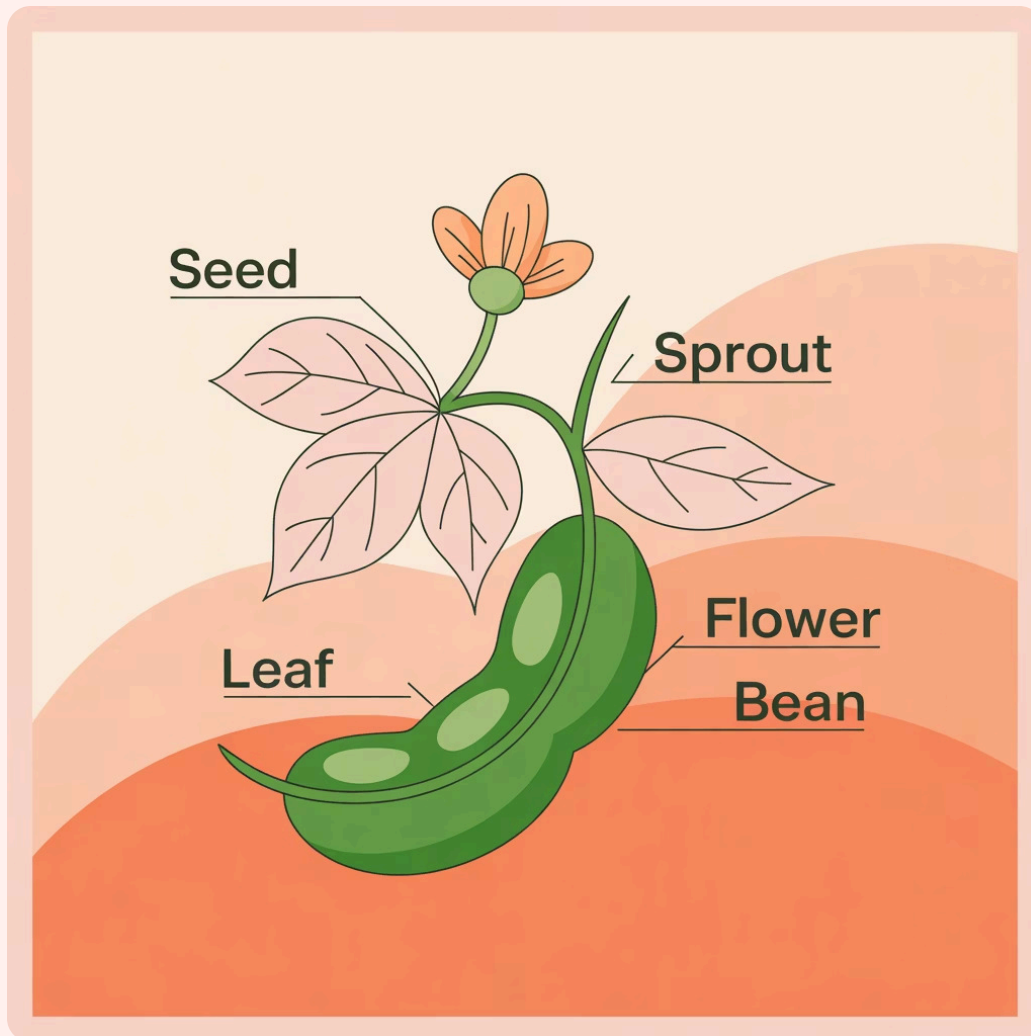
Ciclo de vida de un Bean

Los beans siguen un proceso ordenado desde que Spring los descubre hasta que la aplicación termina:



Registro: Manual vs Automático

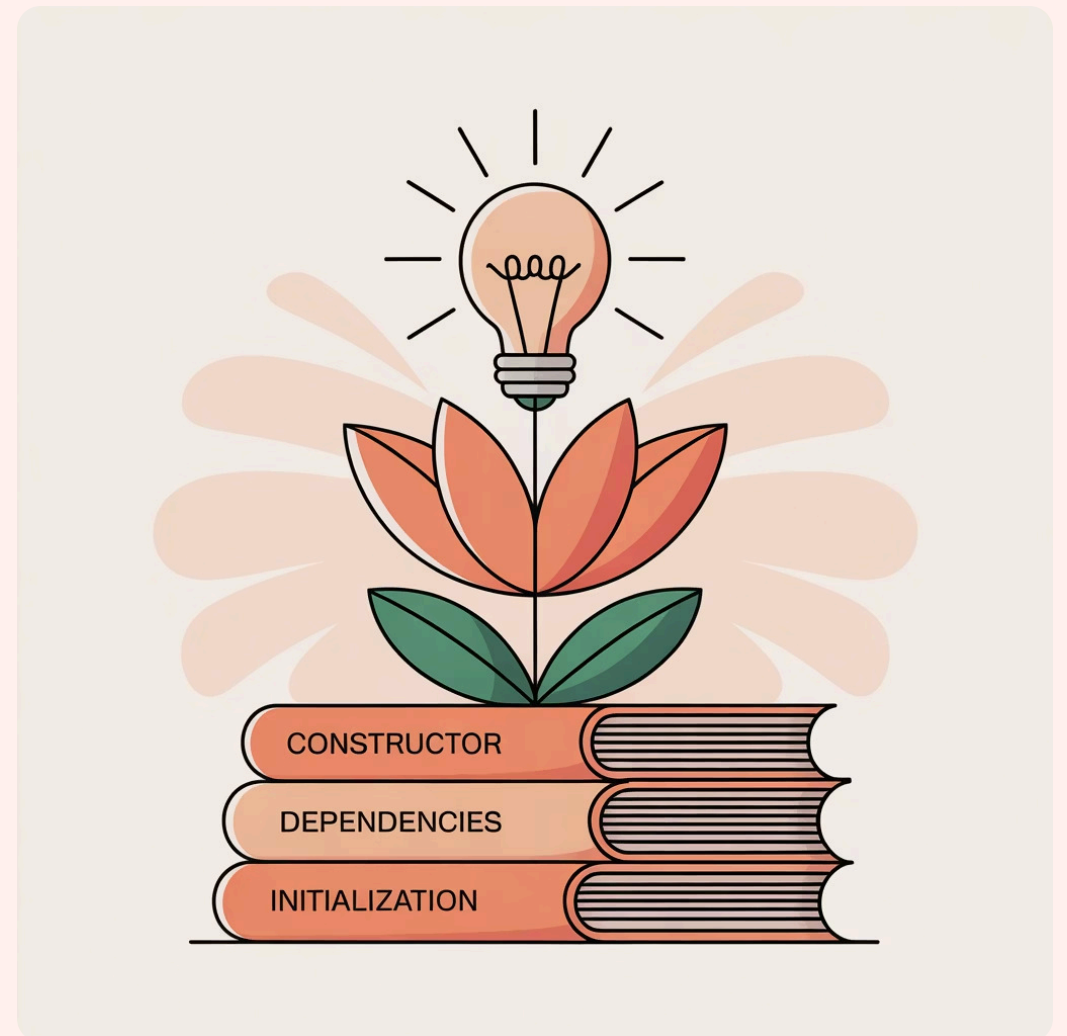
Automático



- Usa anotaciones como `@Component`
- Spring detecta y registra automáticamente
- Más simple y común
- Ideal para la mayoría de casos

```
@Component
public class MiServicio {
    // Spring lo registra solo
}
```

Manual



- Defines beans en clases `@Configuration`
- Más control sobre la creación
- Útil para configuraciones complejas
- Permite personalizar parámetros

```
@Configuration
public class Config {
    @Bean
    public MiServicio miServicio() {
        return new MiServicio();
    }
}
```

¿Por qué son importantes los Beans?



Reutilización

Una sola instancia sirve para toda la aplicación. No desperdicias memoria creando objetos duplicados.



Inyección de Dependencias

Spring conecta automáticamente los objetos que se necesitan entre sí, sin que tengas que hacerlo manualmente.



Gestión Automática

Spring se encarga de crear, configurar y destruir los objetos. Tu código queda más limpio y enfocado.



Facilita Testing

Puedes reemplazar fácilmente beans reales por mocks durante las pruebas unitarias.

Resumen: Lo que has aprendido

1

ApplicationContext

Es el contenedor central que gestiona todos los objetos de tu aplicación Spring

2

Beans

Son objetos que Spring crea y administra automáticamente por ti

3

Component Scan

Spring busca automáticamente clases con anotaciones para registrarlas como beans

4

Ciclo de Vida

Los beans pasan por etapas ordenadas: detección, creación, inyección, uso y destrucción

ApplicationContext

A mind map diagram with 'ApplicationContext' at the center. It has several branches, each with a colored rectangular box. The branches are arranged in a circular pattern around the central node. The colors of the boxes include shades of orange, pink, and light orange. The diagram is set against a background with a subtle pattern of overlapping circles in similar colors.



¡Hora del código!

Demo práctica

Ahora veremos todo esto en acción con ejemplos reales de código



[Guion parte del código en vivo]

- Crear clases con `@Component`
- Mostrar cómo Spring las registra
- Acceder a beans desde `ApplicationContext`
- Verificar el ciclo de vida en acción

Próximos pasos

En los siguientes videos de la serie profundizaremos en:



Inyección de Dependencias

Cómo Spring conecta automáticamente tus beans y las diferentes formas de hacerlo (@Autowired, constructor, setter)



Estereotipos de Spring

Diferencias entre @Component, @Service, @Repository y @Controller - cuándo usar cada uno



Scopes de Beans

Singleton, Prototype y otros alcances - cómo controlar el ciclo de vida de tus objetos



Configuración Avanzada

Profiles, configuración condicional y personalización del comportamiento de Spring



¡Excelente trabajo!

Has dado un paso fundamental en tu camino hacia dominar Spring Boot

Sigue practicando

La práctica hace al maestro. Experimenta con el código del video

Próximo video

Continuaremos con Inyección de Dependencias para completar el panorama

Comunidad

Comparte tus dudas y logros en los comentarios

Java con Spring Boot de 0 a Pro • Nos vemos en el próximo video 🚀